

Assignment-21.5

Name: Arjun Manoj

H. No:2303A52134

Batch:44

Task A – AI-Based Initial Program Development

Prompt:

Generate a basic Python program that calculates the total and average marks of students and assigns grades. Then enhance the program by adding input validation, error handling, and additional features such as grade classification and pass/fail status. Compare both versions and analyze the improvements.

Code:

```
Assignment21.5 task1.py > ...
1  #AI-Based Initial Program Development
2  # Use AI to generate a basic program (e.g., Student Grade Calculator).
3  # Ask AI to enhance it with validation and additional features.
4  # Compare both versions.
5
6
7  #basic
8  # Basic Student Grade Calculator
9
10 name = input("Enter student name: ")
11
12 marks1 = int(input("Enter marks for Subject 1: "))
13 marks2 = int(input("Enter marks for Subject 2: "))
14 marks3 = int(input("Enter marks for Subject 3: "))
15
16 total = marks1 + marks2 + marks3
17 average = total / 3
18
19 # Assign grade
20 if average >= 90:
21     grade = "A"
22 elif average >= 75:
23     grade = "B"
24 elif average >= 50:
25     grade = "C"
26 else:
27     grade = "D"
28
29 print(f"Student: {name}")
30 print(f"Total: {total}")
31 print(f"Average: {average}")
32 print(f"Grade: {grade}")
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 3

```
PS C:\Users\arjun\OneDrive\Desktop\AI - Assist> & C:\Users\arjun\AppData\Local\Programs\Python\Python310\python.exe "c:/Users/arjun/OneDrive/Desktop/AI - Assist/Assignment21.5 task1.py"
Enter student name: arjun
Enter marks for Subject 1: 34
Enter marks for Subject 2: 34
Enter marks for Subject 3: 87
Student: arjun
Total: 155
Average: 51.666666666666664
```

```
Assignment21.5 task1.py > ...
26 else:
27     grade = "Fail"
28
29 print("Student:", name)
30 print("Total:", total)
31 print("Average:", average)
32 print("Grade:", grade)
33
34 #enhanced
35 # Enhanced Student Grade Calculator with Validation
36
37 def get_valid_marks(subject):
38     while True:
39         try:
40             marks = int(input(f"Enter marks for {subject} (0-100): "))
41
42             if 0 <= marks <= 100:
43                 return marks
44             else:
45                 print("Marks must be between 0 and 100.")
46
47         except ValueError:
48             print("Invalid input! Please enter numbers only.")
49
50
51 # Input student name
52
PS C:\Users\arjun\OneDrive\Desktop\AI - Assist> & C:\Users\arjun\AppData\Local\Programs\Python\Python310\python.exe "c:/Users/arjun/OneDrive/Desktop/AI
- Assist/Assignment21.5 task1.py"
Total: 155
Average: 51.666666666666664
Grade: C
Enter student name: 34
Enter marks for Subject 1 (0-100): 34
Enter marks for Subject 2 (0-100): 87
Enter marks for Subject 3 (0-100):
```

```
Assignment21.5 task1.py > ...
51 # Input student name
52 name = input("Enter student name: ")
53
54 # Get validated marks
55 marks1 = get_valid_marks("Subject 1")
56 marks2 = get_valid_marks("Subject 2")
57 marks3 = get_valid_marks("Subject 3")
58
59 # Calculate results
60 total = marks1 + marks2 + marks3
61 average = total / 3
62
63 # Assign grade
64 if average >= 90:
65     grade = "A"
66 elif average >= 75:
67     grade = "B"
68 elif average >= 50:
69     grade = "C"
70 else:
71     grade = "F"
72
73 # Pass/Fail status
74 status = "Pass" if average >= 50 else "Fail"
75
76 # Display results
77 print("\n--- Student Result ---")
78 print("Student Name:", name)
79 print("Total Marks:", total)
80 print("Average Marks:", round(average, 2))
81 print("Grade:", grade)
82 print("Status:", status)
83
```

Observation:

The initial AI-generated program successfully calculated total marks, average, and grade but lacked input validation and error handling, making it prone to crashes and incorrect data entry. The enhanced version improved reliability by adding input validation, range checking (0–100), and exception handling to prevent invalid inputs. Additional features such as pass/fail status and formatted output made the program

more user-friendly and robust. Overall, the enhanced version is safer, more reliable, and suitable for real-world usage.

Task B – AI-Assisted Application Improvement

Prompt:

Generate a simple Python Library Management program that allows adding and viewing books. Then enhance the program by adding new functionalities such as searching books, deleting books, and improving program structure using functions. Analyze how the improvements affect the structure and performance of the program.

Code:

```
Assignment21.5 task2.py >...
1  #AI-Assisted Application Improvement
2  #Generate a simple program (e.g., Library Management).
3  #Use AI to add new functionalities.
4  #Analyze improvements in structure and performance.
5
6  #basic
7  # Basic Library Management System
8
9  library = []
10
11 while True:
12     print("\n1. Add Book")
13     print("2. View Books")
14     print("3. Exit")
15
16     choice = input("Enter choice: ")
17
18     if choice == "1":
19         book = input("Enter book name: ")
20         library.append(book)
21         print("Book added successfully.")
22
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 3
3. Exit
Enter choice: 1
Enter book name: dsd
Book added successfully.

1. Add Book
2. View Books
3. Exit
Enter choice: 2
Books in Library:
dsd

1. Add Book
2. View Books
```

```
Assignment21.5 task2.py > ...
23     elif choice == "2":
24         print("Books in Library:")
25         for book in library:
26             print(book)
27
28     elif choice == "3":
29         print("Exiting program.")
30         break
31
32     else:
33         print("Invalid choice.")
34
35 #enhanced
36 # Enhanced Library Management System
37
38 library = []
39
40
41 # Function to add book
42 def add_book():
43     book = input("Enter book name: ")
44
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 3
Enter book name: dsd
Book added successfully.

1. Add Book
2. View Books
3. Exit
Enter choice: 2
Books in Library:
dsd

1. Add Book
2. View Books
3. Exit
Enter choice: 1
```

```
Assignment21.5 task2.py > ...
53 def view_books():
54     if not library:
55         print("Library is empty.")
56     else:
57         print("\nBooks in Library:")
58         for index, book in enumerate(library, start=1):
59             print(index, ".", book)
60
61
62 # Function to search book
63 def search_book():
64     book = input("Enter book name to search: ")
65
66     if book in library:
67         print("Book found in library.")
68     else:
69         print("Book not found.")
70
71
72 # Function to delete book
73 def delete_book():
74     book = input("Enter book name to delete: ")
75
76     if book in library:
77         library.remove(book)
78         print("Book deleted successfully.")
79     else:
80         print("Book not found.")
81
82
83 # Main menu loop
84 while True:
85     print("\n==== Library Menu =====")
86     print("1. Add Book")
87     print("2. View Books")
88     print("3. Search Book")
89     print("4. Delete Book")
```

```

82
83 # Main menu loop
84 while True:
85     print("\n==== Library Menu =====")
86     print("1. Add Book")
87     print("2. View Books")
88     print("3. Search Book")
89     print("4. Delete Book")
90     print("5. Exit")
91
92     choice = input("Enter choice: ")
93
94     if choice == "1":
95         add_book()
96
97     elif choice == "2":
98         view_books()
99
100    elif choice == "3":
101        search_book()
102
103    elif choice == "4":
104        delete_book()
105
106    elif choice == "5":
107        print("Exiting program.")
108        break
109
110    else:
111        print("Invalid choice.")

```

Observation:

The initial AI-generated Library Management program supported only basic operations such as adding and viewing books, but lacked advanced features and proper structure. The improved version introduced new functionalities like searching and deleting books, making the system more useful and interactive. Functions were added to organize the code into modular sections, improving readability, maintainability, and scalability. Overall, the enhanced program demonstrates better structure, improved performance efficiency, and easier expansion for future features such as file storage or database integration.

Task C – AI for Commit Message Writing

Prompt:

Perform multiple code changes in a Python project such as adding validation, fixing errors, and improving features. Use AI to generate meaningful commit messages for each change. Compare the AI-generated commit messages with manually written commit messages and analyze their clarity and usefulness.

Code:

```
Assignment21.5 atsk3.py > ...
1  #AI for Commit Message Writing
2  #Perform multiple code changes.
3  #Use AI to generate commit messages.
4  #Compare with manually written messages.
5
6  from dbm.ndbm import library
7
8
9  def get_marks():
10     while True:
11         try:
12             marks = int(input("Enter marks (0-100): "))
13             if 0 <= marks <= 100:
14                 return marks
15             else:
16                 print("Marks must be between 0 and 100.")
17         except ValueError:
18             print("Invalid input.")
19
20  def search_book(library):
21     book = input("Enter book name to search: ")
22
23     if book in library:
24         print("Book found.")
25     else:
26         print("Book not found.")
27
28  print("\n==== Library Records =====")
29  for i, book in enumerate(library, start=1):
30     print(i, ".", book)
```

Observation:

AI-generated commit messages were more descriptive and clearly explained the purpose of each code change, making them easier to understand during project tracking. Manually written messages were shorter but less informative, which could make it harder to identify specific changes later. Overall, AI-assisted commit message writing improves clarity, consistency, and maintainability in version control workflows.

Task D – Pair Programming with AI Assistance

Prompt:

Student A develops an initial Python program (such as a basic calculator or library system). Student B uses AI assistance to enhance and optimize the program by adding validation, improving structure, and adding new features. Integrate the improved code using Git by committing and merging the changes. Analyze how AI-assisted pair programming improves code quality and teamwork.

Code:

```
Assignment21.5 task4.py > ...
1  #Pair Programming with AI Assistance
2  # Student A develops initial code.
3  # Student B uses AI to enhance and optimize.
4  # Integrate changes using Git.
5
6  #Student A
7  # Student A: Basic Calculator Program
8
9  num1 = float(input("Enter first number: "))
10 num2 = float(input("Enter second number: "))
11
12 operation = input("Enter operation (+, -, *, /): ")
13
14 if operation == "+":
15     print("Result:", num1 + num2)
16
17 elif operation == "-":
18     print("Result:", num1 - num2)
19
20 elif operation == "*":
21     print("Result:", num1 * num2)
22
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 3

```
PS C:\Users\arjun\OneDrive\Desktop\AI - Assist> python Assignment21.5 task4.py
C:\Users\arjun\miniforge3\condabin\python.exe: can't open file 'C:\Users\arjun\OneDrive\Desktop\AI - Assist\Assig
file or directory
❖ PS C:\Users\arjun\OneDrive\Desktop\AI - Assist> python "Assignment21.5 task4.py"
Enter first number: 2
Enter second number: 2
Enter operation (+, -, *, /): +
Result: 4.0
Enter first number: █
```

```

Assignment21.5 task4.py > ...
20 elif operation == "**":
21     print("Result:", num1 * num2)
22
23 elif operation == "/":
24     print("Result:", num1 / num2)
25
26 else:
27     print("Invalid operation")
28
29 #Student B
30 # Student B: Enhanced Calculator with Validation
31
32 def get_number(prompt):
33     while True:
34         try:
35             return float(input(prompt))
36         except ValueError:
37             print("Invalid input! Please enter a number.")
38
39
40 def calculate(num1, num2, operation):
41     if operation == "+":
42         return num1 + num2
43
44     elif operation == "-":
45         return num1 - num2
46
47     elif operation == "**":
48         return num1 * num2
49
50     elif operation == "/":
51         if num2 == 0:
52             return "Error: Division by zero not allowed"
53         return num1 / num2
54
55     else:
56         return "Invalid operation"

```

```

elif operation == "/":
    if num2 == 0:
        return "Error: Division by zero not allowed"
    return num1 / num2

else:
    return "Invalid operation"

```

```

# Main program
num1 = get_number("Enter first number: ")
num2 = get_number("Enter second number: ")

operation = input("Enter operation (+, -, *, /): ")

result = calculate(num1, num2, operation)

print("Result:", result)

```

Observation:

Student A created a basic working program, while Student B used AI assistance to enhance the code with input validation, modular functions, and error handling. Using Git allowed both versions to be managed efficiently through branching and merging. AI-assisted pair programming improved code quality, reduced errors, and encouraged better collaboration and structured development.

Task E – Collaborative Development using Branches

Prompt:

Divide the development of a Python project between two students using Git branches. Student A implements the basic functionality, while Student B uses AI assistance to add new features. Each student works on a separate branch, and the contributions are merged into a single final project. Analyze how branching supports collaborative development and feature integration.

Code:

```
Assignment21.5 task5.py > search_task
1  #Divide work between two students.
2  # Use AI for feature implementation.
3  # Merge contributions into a single project.
4
5  # Student A# Student A: Basic To-Do List
6
7  tasks = []
8
9  def add_task():
10     task = input("Enter new task: ")
11     tasks.append(task)
12     print("Task added successfully.")
13
14  def view_tasks():
15     print("\nYour Tasks:")
16     for i, task in enumerate(tasks, start=1):
17         print(i, ".", task)
18
19  while True:
20     print("\n1. Add Task")
21     print("2. View Tasks")
22     print("3. Exit")

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 3
PS C:\Users\arjun\OneDrive\Desktop\AI - Assist> python "Assignment21.5 task5.py"

1. Add Task
2. View Tasks
3. Exit
Enter choice: 1
Enter new task: 23
Task added successfully.

1. Add Task
2. View Tasks
3. Exit
Enter choice: 
```

```

19 while True:
20     print("\n1. Add Task")
21     print("2. View Tasks")
22     print("3. Exit")
23
24     choice = input("Enter choice: ")
25
26     if choice == "1":
27         add_task()
28
29     elif choice == "2":
30         view_tasks()
31
32     elif choice == "3":
33         print("Exiting program.")
34         break
35
36     else:
37         print("Invalid choice.")
38
39 # Student B: Enhanced To-Do List with Priorities
40 # Student B: Additional Features
41
42 def delete_task(tasks):
43     view_tasks(tasks)
44
45     task_no = int(input("Enter task number to delete: "))
46
47     if 1 <= task_no <= len(tasks):
48         removed = tasks.pop(task_no - 1)
49         print("Deleted task:", removed)
50     else:
51         print("Invalid task number.")
52
53
54 def search_task(tasks):
55     keyword = input("Enter keyword to search: ")

```

```

52
53
54 def search_task(tasks):
55     keyword = input("Enter keyword to search: ")
56
57     found = False
58
59     for task in tasks:
60         if keyword.lower() in task.lower():
61             print("Found:", task)
62             found = True
63
64     if not found:
65         print("No matching task found.")

```

Observation:

Dividing work into separate branches allowed Student A and Student B to develop features independently without affecting the main program. AI assistance helped Student B quickly implement additional features such as delete and search functions. Merging branches combined both contributions into a single project, improving collaboration, reducing

conflicts, and making the development process more organized and efficient.